

# Практический кейс: «Тиммейт-Детектор: ML для поиска руинеров в Dota 2»

## Контекст

Киберспортивная организация готовится к квалификациям на The International. У них есть данные о 100 000 матчей пабликов и тренировочных игр. Проблема: в командах постоянно попадают **руинеры** — игроки, которые намеренно или случайно портят игру: фидят, не покупают варды, плачут пятого керри, уходят AFK на 10-й минуте.

Скаут-отдел тратит кучу времени на просмотр реплеев. Они хотят ML-модель, которая по данным матча определяет **вероятность того, что игрок — руинер**. И главное: модель должна **объяснять**, почему она так решила — чтобы тренер за 10 секунд понял, что сказать игроку на разборе.

## Роль студента

Вы — Junior Data Scientist в аналитическом отделе киберспортивной организации. Вам нужно построить прототип модели, которая находит руинеров, и защитить её перед «главным тренером» (лабником).

## Вводные данные

Синтетический датасет на основе открытых данных Dota 2 или сгенерированный. Признаки выбирает студент.

**Дисбаланс классов: ~5% руинеров.**

## Задания для студентов (Этапы решения)

### Этап 1. Разведочный анализ (EDA) и работа с дисбалансом (15%)

Проанализировать распределение целевого признака и числовых признаков.

**Вопрос для подумать:** Правда ли, что все руинеры — это игроки с низким GPM? А если у человека просто плохой матч? Как отличить «плохую игру» от «руинерства»?

Предложить стратегию работы с дисбалансом. Обосновать: SMOTE, class weights, пороговая классификация или что-то ещё. **Важно:** не использовать SMOTE слепо —

подумать, какие признаки синтетически генерировать нельзя (например, `afk_time` не может быть отрицательным).

## Этап 2. Генерация признаков (Feature Engineering) (20%)

Создать минимум 3 новых признака, которые помогут модели. Примеры:

- **Ward Efficiency** =  $\text{wards\_placed} / (\text{net\_worth} / 1000)$  — сколько вардов на тысячу золота. Руинеры часто экономят на вардах.
- **AFK Ratio** =  $\text{afk\_time} / \text{duration}$  — доля времени в AFK.
- **Role Mismatch** = флаг, если игрок на позиции 5 купил предметы для керри (например, Battle Fury на 20-й минуте).
- **Smoke per Minute** =  $\text{smoke\_uses} / (\text{game\_duration} / 60)$ .

**Бизнес-смысл:** тренеру важно не просто «модель сказала руинер», а «модель сказала руинер, потому что у него 0 вардов за 40 минут и AFK 15 минут».

## Этап 3. Моделирование и выбор метрик (25%)

Обучить базовую модель (Логистическая регрессия) и бустинг (CatBoost / XGBoost / LightGBM).

**Важно:** не использовать Accuracy и ROC-AUC как основные метрики. Обосновать выбор **PR-AUC (Average Precision)** или **F2-score**.

**Почему F2?** Потому что для тренера важнее не пропустить руинера (Recall), чем ошибочно обвинить нормального игрока (Precision). Но и совсем без Precision нельзя — иначе модель будет называть руинерами всех.

## Этап 4. Бизнес-оптимизация (Cost-Sensitive Learning) (20%)

Матрица издержек:

- **False Positive (ложное обвинение):** Тренер тратит время на разбор невиновного игрока (30 минут = 1000 руб.) + игрок обижается и уходит из команды (упущенная выгода 10 000 руб.). **Итого: -11 000 руб.**
- **False Negative (пропуск руинера):** Команда проигрывает важный матч из-за руинера. Потеря призовых и репутации. **Итого: -50 000 руб.** (или  $\text{match\_prize} * 1.5$ ).
- **True Positive:** тренер вовремя разобрался, руинера кикнули, команда выиграла. **+5 000 руб.**
- **True Negative:** всё хорошо. **0 руб.**

**Задача:** найти оптимальный порог отсеечения вероятности, при котором суммарный финансовый убыток организации минимален. Построить график зависимости убытка от порога.

**Вопрос для подумать:** что если руинер — это тиммейт, который просто играет плохо, но старается? Как модель должна это учитывать? (Ответ: это не руинер, это низкий skill. Модель должна различать.)

## Этап 5. Интерпретируемость (Explainable AI) (20%)

Бизнес требует объяснимости.

1. С помощью **SHAP** (или LIME) рассчитать вклад признаков для 5 конкретных игроков из тестовой выборки, которых модель пометила как руинеров (высокий  $FP$  или  $TP$ ).
2. Сформулировать «Код причины» для тренера. Например:
  - *«Код 322:  $AFK > 20\%$  времени + 0 вардов за игру + 15 смертей за 30 минут»*
  - *«Код 404: Role Mismatch — позиция 5 купил Battle Fury на 25-й минуте»*
  - *«Код 228: Смерти в первые 10 минут  $> 10$  + курьер не убит ни разу»*

**Важно:** код должен быть понятен тренеру без ML-образования. И желательно смешной, чтобы разрядить обстановку на разборе.

## Формат сдачи и критерии оценки

Студенты сдают работу в формате **Jupyter Notebook** и **Презентации для бизнеса** (на 5–7 минут).

### Критерии оценки (100 баллов):

1. **Код и ML-пайплайн (30 б):** чистота кода, корректность кросс-валидации (использование `TimeSeriesSplit` или стратификации, чтобы не было утечек из будущего), адекватная работа с дисбалансом.
2. **Бизнес-логика (30 б):** правильный расчёт матрицы издержек, нахождение оптимального порога, перевод метрик ML на язык денег (рубли).
3. **Интерпретируемость (20 б):** корректное применение SHAP, понятные выводы для бизнеса.
4. **Защита/Презентация (20 б):** умение объяснить нетехническому заказчику (лабнику в роли главного тренера), почему выбрана именно эта модель и как она сэкономит деньги и нервы.

## Бонус: идеи для «Кода причины»

- « $AFK > 20\%$  времени. Подозрение на просмотр VK Video во время игры.»
- «Role Mismatch. Позиция 5 купил Battle Fury. Тренер, поговорите с ним.»
- «0 вардов за 40 минут. Даже у крипов есть варды. А у него нет.»
- «Байбэков больше, чем смертей. Как? Просто как?»